

SalaryBox Mobile App Developer - What to Learn

MUST Learn (Class A keywords with no direct project experience)

- **Kotlin and/or Swift (native mobile)** – your production mobile experience is React Native (JavaScript/TypeScript), not native Kotlin (Android) or Swift (iOS). This JD wants native development specifically, which is a real language and paradigm switch. Bridge from your React Native background: you already understand the platform lifecycle, permissions, and native module concepts from working with RN's bridge – Kotlin/Swift is the "no bridge needed" version of the same platform APIs. Build one small native app (start with whichever platform you use daily) to get comfortable with the syntax and tooling (Android Studio / Xcode) before interviewing.
- **Jetpack Compose and/or SwiftUI** – modern declarative UI frameworks for native Android/iOS, conceptually close to React's component model (declarative UI driven by state) – this is actually your fastest bridge point, since you already think in components and state. Compose uses Kotlin, SwiftUI uses Swift; once you know the base language, the UI framework itself should feel familiar.
- **Coroutines/Flow (Kotlin) or async/await (Swift)** – native concurrency primitives, parallel to JavaScript's Promises/async-await that you already use daily. Kotlin Flow is reactive-stream-like (comparable conceptually to Redux/MobX observables you've used); Swift's async/await is very close syntactically to JS async/await, making Swift concurrency the easier of the two to pick up quickly.
- **Retrofit/Ktor (Android) or URLSession (iOS)** – native HTTP client libraries, the platform-specific equivalent of the REST API calls you already make from React Native (via fetch/axios). Retrofit in particular has a declarative, interface-based API design that should feel approachable once you've seen it once.

GOOD to Learn (strengthens the application, not blocking)

- **Xcode Instruments / Android Profiler, applied natively** – you have real crash/performance debugging experience (Sentry, DevTools at Clink AI), but not with the native platform-specific profiling tools. The debugging mindset transfers directly; the tool surface is new.
- **Deep links, background execution, push notifications (native)** – you've likely touched these conceptually via Expo/React Native's abstractions; the native implementation (platform-specific manifest/plist configuration) is more manual but the underlying concepts (intent filters, background modes, APNs/FCM) are the same.

Fast-Ramp Note

This JD explicitly wants native Kotlin/Swift development, which is a genuine language and platform switch from your React Native background – not an incremental stretch. The one area that IS a strong, direct match: AI-assisted engineering fluency (Claude Code, Cursor, strong code-review judgment on AI-generated code) is something you already do daily and genuinely, and this JD calls it out explicitly as a requirement alongside the native skills. Lead with that fluency and your shipping/release-ownership track record (EAS builds, Play Console/App Store submissions, crash triage) while being upfront that native Kotlin/Swift is new ground you're picking up.